# UART RTL Design Handbook

*Verilog RTL Implementation, Verification, and Documentation*
*UART-RTL-FPGA*

*Author : REVANTH ROY NAIDU TADIKONDA*

*Designation : INTERN@RCI_DRDO*

## 1. Project Overview

UART (Universal Asynchronous Receiver Transmitter) is a serial communication protocol used to exchange data between digital systems without a shared clock. Each side derives its own sampling clock from a known baud rate, and framing bits (start/stop) keep both ends synchronized on a byte-by-byte basis.

This project implements a synthesizable UART Transmitter and Receiver in Verilog RTL, a configurable baud rate generator, a top-level integration wrapper, and a self-checking loopback testbench verified in Vivado simulation.

### Key Parameters

| Parameter | Value |
|---|---|
| **Clock Frequency** | 100 MHz |
| **Baud Rate** | 9600 |
| **Data Width** | 8 bits |
| **Oversampling (RX)** | 16x |
| **Frame Format** | 1 start, 8 data, 1 stop, no parity |

### Features

- Verilog RTL design (synthesizable)
- UART Transmitter (FSM based)
- UART Receiver (16x oversampled)
- Configurable Baud Rate Generator
- 8-bit UART frame, 9600 baud
- Vivado functional simulation
- Self-checking loopback testbench

## 2. Code Review and Corrections

All four modules were reviewed against standard UART RTL practice and cross-checked against the provided simulation waveform. Summary verdict:

| Module | Status | Notes |
|---|---|---|
| **baud_rate_generator.v** | Correct | Counter thresholds (10416 for 9600 baud TX, 651 for 16x RX oversampling at 100 MHz) are accurate. |
| **transmitter.v** | Correct | Clean 4-state FSM. tx_serial_out and busy behave as expected in the waveform (busy high through |

| | | |
|---|---|---|
| | | START/DATA/STOP). |
| **receiver.v** | Correct | Mid-bit sampling at sample_cnt==8 and bit-period rollover at sample_cnt==15 correctly implement 16x oversampling. Verified against waveform: shift_reg fills 00->01->05->25->...->a5. |
| **uart_top.v** | Correct | Loopback wiring (rx_in = tx_serial_out) is appropriate for the self-checking testbench. |
| **uart_tb.v** | Correct | wait(ready) plus comparison against expected byte is a sound self-checking method. |

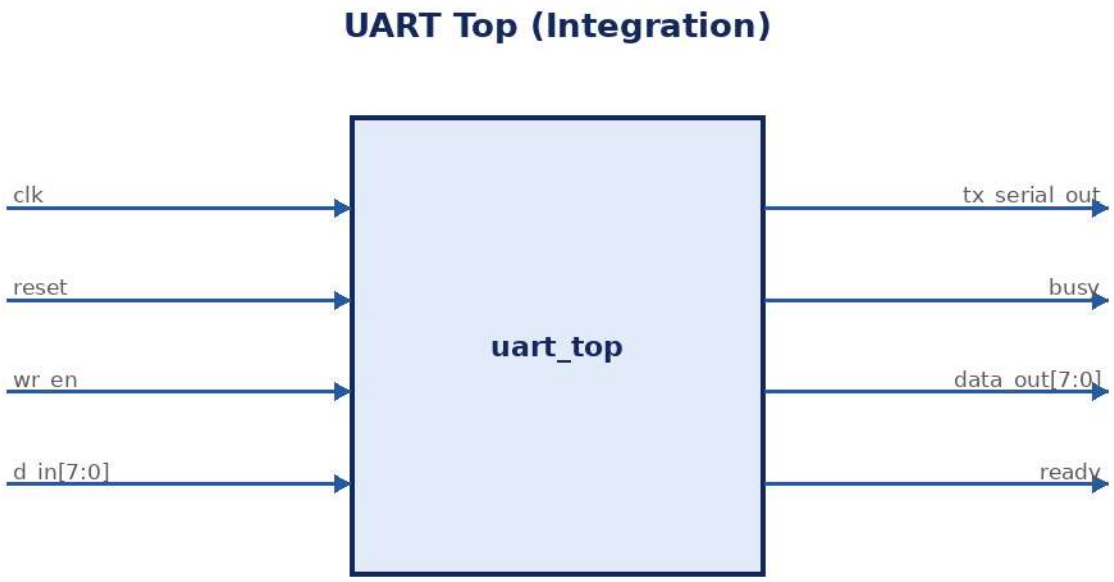
No functional bugs were found. The waveform confirms correct operation: d_in = A5 is serialized, sampled at 16x, reassembled in shift_reg, and reported on data_out as A5 when ready pulses; the same is repeated correctly for 3C.

### Minor Style Recommendations (optional, non-functional)

- In receiver.v, declare state encodings as a 2-bit Verilog 'state' enum comment matching transmitter.v's parameter style for consistency (both currently work; receiver uses localparam, transmitter uses parameter).
- Add a synchronous reset value for 'state' output port comment so reviewers can see encodings without opening the file.
- Consider adding a parity bit and a configurable BAUD_DIV parameter for future reuse (see Future Work).

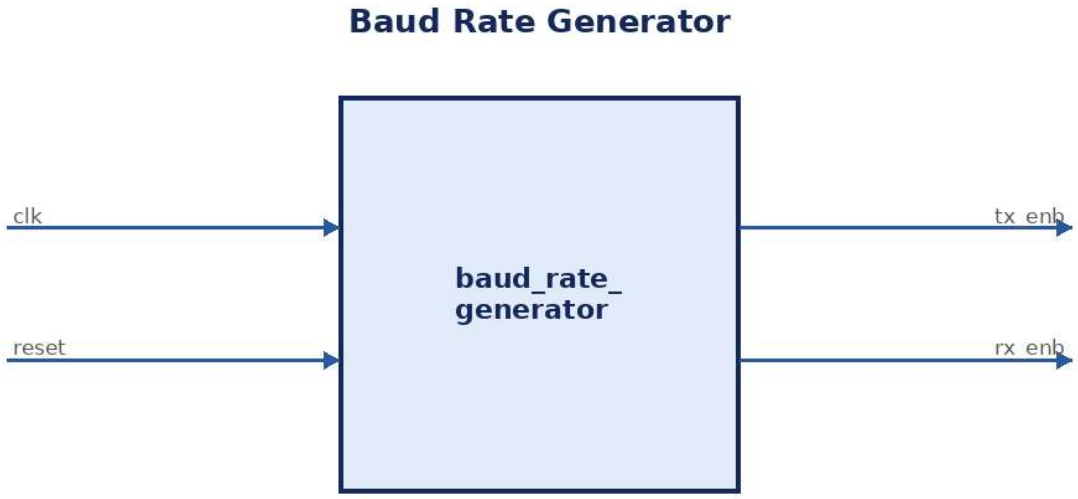
## 3. System Architecture

uart_top integrates three blocks: the baud rate generator produces tx_enb and rx_enb strobes; the transmitter serializes parallel data on tx_enb pulses; the receiver deserializes the incoming bit stream on rx_enb pulses (16x oversampled). In the testbench, rx_in is looped back from tx_serial_out for self-checking.



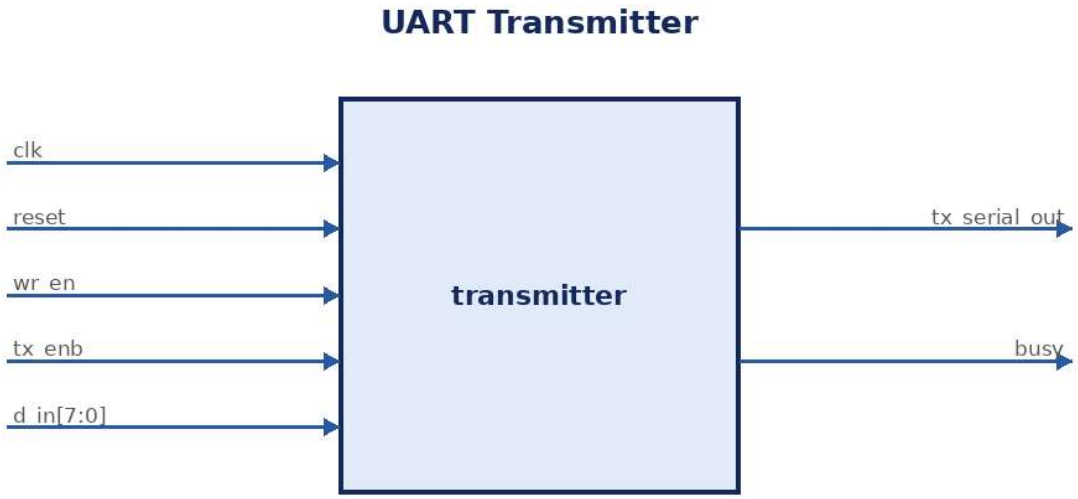

## 4. Module Block Diagrams

### 4.1 baud_rate_generator



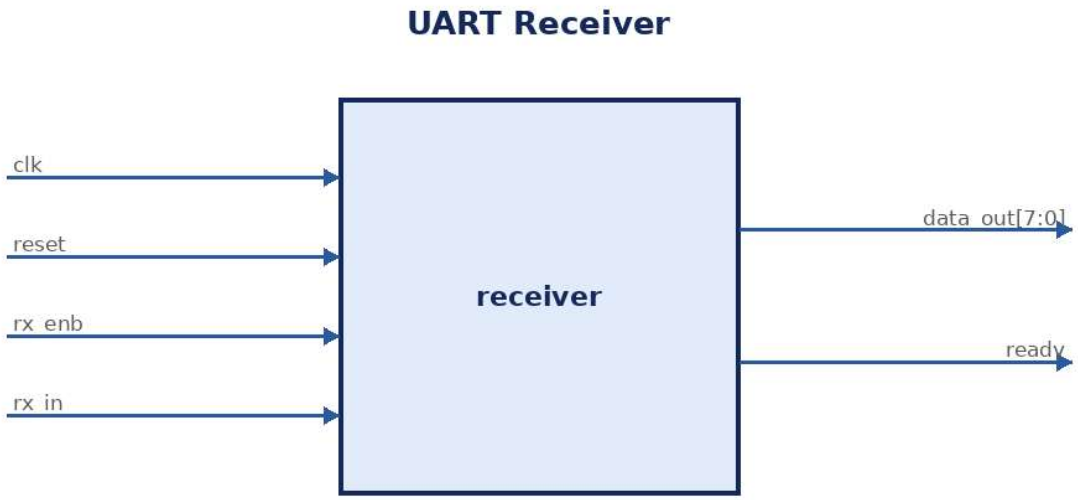

Generates two enable strobes from the 100 MHz system clock: tx_enb pulses once every 10417 clock cycles (9600 baud) and rx_enb pulses once every 652 clock cycles (16x 9600 baud), used by the receiver for oversampling.

## 4.2 transmitter




Converts an 8-bit parallel word (d_in) into a serial bit stream (tx_serial_out) framed with a start bit (0) and a stop bit (1), gated by tx_enb. busy is asserted from the start bit through the stop bit.

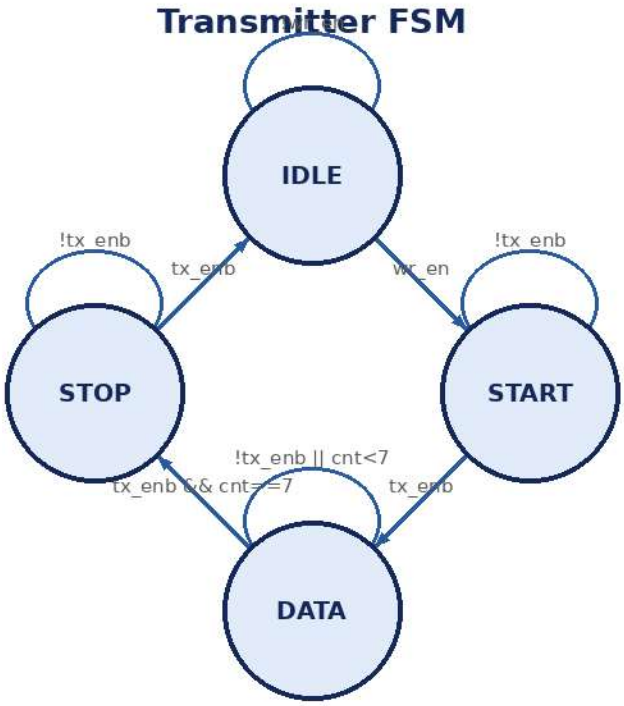
## 4.3 receiver




Converts the serial bit stream (rx_in) back into an 8-bit parallel word (data_out), oversampling each bit 16 times via rx_enb and sampling at the bit center (sample_cnt==8). ready pulses for one clock cycle when a valid byte has been received.

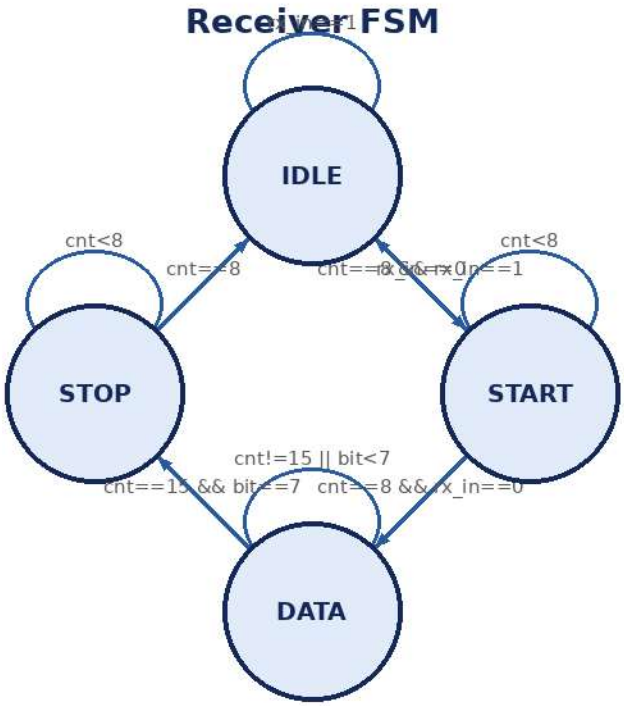
## 5. Finite State Machine Diagrams

### 5.1 Transmitter FSM




| State | Behavior |
|---|---|
| **IDLE** | tx_serial_out=1, busy=0. On wr_en, latch d_in and move to START. |
| **START** | tx_serial_out=0 (start bit), busy=1. On tx_enb, move to DATA. |
| **DATA** | tx_serial_out=data[count], busy=1. On tx_enb, increment count; after bit 7, move to STOP. |
| **STOP** | tx_serial_out=1 (stop bit), busy=1. On tx_enb, return to IDLE. |

### 5.2 Receiver FSM




| State | Behavior |
|---|---|
| **IDLE** | Waits for rx_in to fall to 0 (start bit edge); resets counters. |
| **START** | Counts 8 rx_enb pulses (half bit period) then re-checks rx_in==0 to confirm a valid start bit; otherwise returns to IDLE (glitch rejection). |
| **DATA** | Samples rx_in at sample_cnt==8 (bit center) into shift_reg[bit_cnt]; advances to next bit at sample_cnt==15; after 8 bits, moves to STOP. |
| **STOP** | Waits 8 rx_enb pulses, checks rx_in==1 for a valid stop bit, latches shift_reg into data_out and pulses ready for one clock. |

## 6. Baud Rate Calculation

With a 100 MHz system clock and a target baud rate of 9600:

```
TX divisor   = Clock / Baud         = 100,000,000 / 9,600       = 10,416
RX divisor   = Clock / (Baud x 16)  = 100,000,000 / (9,600 x 16) = 651
```

The RX divisor is 16x smaller so the receiver can sample each incoming bit 16 times for mid-bit alignment and noise rejection.

## 7. UART Frame Format

```
Idle(1) -> Start(0) -> Bit0 -> Bit1 -> Bit2 -> Bit3 -> Bit4 -> Bit5 -> Bit6 -> Bit7
-> Stop(1) -> Idle(1)
```

Total frame length = 10 bit periods (1 start + 8 data + 1 stop), LSB first.

## 8. Simulation Waveform Analysis

The provided Vivado simulation waveform was reviewed in detail and confirms correct functional behavior of the full loopback path:

- d_in = A5 is loaded with wr_en pulsed; tx_serial_out serializes the start bit, then data bits, then stop bit while busy stays high.
- rx_enb toggles continuously (16x oversampling clock); the receiver's bit_cnt increments 0->7 as shift_reg accumulates 00 -> 01 -> 05 -> 25 -> ... -> A5.
- data_out updates to A5 and ready pulses once the stop bit is validated, matching the testbench's expected value for Test Vector 1.
- The same sequence repeats correctly for the second test vector (3C), confirming the FSMs reset cleanly between frames.

This is consistent with a fully passing self-checking testbench (PASS : A5 Received, PASS : 3C Received, ...).

## 9. Functional Verification Checklist

- Reset behavior (all registers clear correctly)
- Single byte transmit/receive (A5)
- Multiple consecutive bytes (3C, 81, 55)
- Full loopback (tx_serial_out -> rx_in)
- Start bit detection and glitch rejection
- Stop bit validation
- busy signal timing
- ready signal (one-cycle pulse) timing

## 10. Recommended GitHub Repository Structure

```
UART-RTL-FPGA
|-- README.md
|-- docs
|   |-- UART_Architecture.pdf
|   |-- UART_BlockDiagram.png
|   |-- UART_FSM_TX.png
|   |-- UART_FSM_RX.png
|   |-- BaudRateCalculation.pdf
|   `-- Waveforms/
```

```
|-- rtl
|   |-- baud_rate_generator.v
|   |-- transmitter.v
|   |-- receiver.v
|   `-- uart_top.v
|-- tb
|   `-- uart_tb.v
|-- simulation
|   |-- tx_waveform.png
|   |-- rx_waveform.png
|   `-- loopback_waveform.png
|-- reports
|   |-- Functional_Verification.pdf
|   `-- Design_Report.pdf
`-- LICENSE
```

## 11. Future Improvements

- Parity bit (even/odd) support
- Configurable baud rate via parameter/register
- TX/RX FIFO integration
- Interrupt support on ready/error
- AXI4-Lite register interface for SoC integration
- DMA support for bulk transfers
- Asynchronous FIFO for clock-domain crossing
- Validation on physical FPGA board (Zynq-7020)

## 12. Skills Demonstrated

- RTL design (Verilog HDL)
- FSM design and state encoding
- Baud rate generation and clock division
- Functional verification and self-checking testbenches
- Vivado simulation and waveform analysis
- Protocol-level understanding (UART framing)
- Technical documentation

## 13. Appendix: Full RTL Source

### 13.1 baud_rate_generator.v

```
`timescale 1ns / 1ps
module baud_rate_generator(
    input clk,
    input reset,
    output reg tx_enb,
```

```
    output reg rx_enb
);
reg [13:0] tx_count;
reg [9:0]  rx_count;
always @(posedge clk)
begin
    if(reset)
    begin
        tx_count <= 14'd0;
        rx_count <= 10'd0;
        tx_enb <= 1'b0;
        rx_enb <= 1'b0;
    end
    else
    begin
        if(tx_count == 14'd10416)
        begin
            tx_count <= 14'd0;
            tx_enb <= 1'b1;
        end
        else
        begin
            tx_count <= tx_count + 1'b1;
            tx_enb <= 1'b0;
        end
        if(rx_count == 10'd651)
        begin
            rx_count <= 10'd0;
            rx_enb <= 1'b1;
        end
        else
        begin
            rx_count <= rx_count + 1'b1;
            rx_enb <= 1'b0;
        end
    end
end
endmodule
```

## 13.2 transmitter.v

```
`timescale 1ns / 1ps
module transmitter(
    input clk,
    input reset,
    input wr_en,
    input tx_enb,
    input [7:0] d_in,
    output reg tx_serial_out,
    output reg busy
);
parameter IDLE  = 2'b00;
```

```
parameter START = 2'b01;
parameter DATA  = 2'b10;
parameter STOP  = 2'b11;
reg [1:0] state;
reg [7:0] data;
reg [2:0] count;
always @(posedge clk)
begin
    if(reset)
    begin
        state         <= IDLE;
        data          <= 8'd0;
        count         <= 3'd0;
        tx_serial_out <= 1'b1;
        busy          <= 1'b0;
    end
    else
    begin
        case(state)
        IDLE :
        begin
            tx_serial_out <= 1'b1;
            busy <= 1'b0;
            if(wr_en)
            begin
                data  <= d_in;
                count <= 3'd0;
                state <= START;
            end
        end
        START :
        begin
            tx_serial_out <= 1'b0;
            busy <= 1'b1;
            if(tx_enb)
            begin
                state <= DATA;
            end
        end
        DATA :
        begin
            busy <= 1'b1;
            tx_serial_out <= data[count];
            if(tx_enb)
            begin
                if(count == 7)
                begin
                    state <= STOP;
                    count <= 0;
                end
                else
                begin
                    count <= count + 1;
```

```
                    state <= DATA;
                end
            end
        end
        STOP :
        begin
            tx_serial_out <= 1'b1;
            busy <= 1'b1;
            if(tx_enb)
            begin
                state <= IDLE;
            end
        end
        default :
        begin
            state <= IDLE;
        end
        endcase
    end
end
endmodule
```

## 13.3 receiver.v

```
`timescale 1ns / 1ps
module receiver(
    input        clk,
    input        reset,
    input        rx_enb,
    input        rx_in,
    output reg [7:0] data_out,
    output reg       ready
);
localparam IDLE  = 2'b00;
localparam START = 2'b01;
localparam DATA  = 2'b10;
localparam STOP  = 2'b11;
reg [1:0] state;
reg [3:0] sample_cnt;
reg [2:0] bit_cnt;
reg [7:0] shift_reg;
always @(posedge clk)
begin
    if(reset)
    begin
        state      <= IDLE;
        sample_cnt <= 4'd0;
        bit_cnt    <= 3'd0;
        shift_reg  <= 8'd0;
        data_out   <= 8'd0;
        ready      <= 1'b0;
    end
```

```
else
begin
    ready <= 1'b0;
    if(rx_enb)
    begin
        case(state)
        IDLE:
        begin
            sample_cnt <= 0;
            bit_cnt    <= 0;
            if(rx_in == 1'b0)
                state <= START;
        end
        START:
        begin
            if(sample_cnt == 4'd8)
            begin
                if(rx_in == 1'b0)
                begin
                    sample_cnt <= 0;
                    state <= DATA;
                end
                else
                begin
                    state <= IDLE;
                end
            end
            else
            begin
                sample_cnt <= sample_cnt + 1'b1;
            end
        end
        DATA:
        begin
            if(sample_cnt == 4'd8)
            begin
                shift_reg[bit_cnt] <= rx_in;
            end
            if(sample_cnt == 4'd15)
            begin
                sample_cnt <= 0;
                if(bit_cnt == 3'd7)
                begin
                    bit_cnt <= 0;
                    state   <= STOP;
                end
                else
                begin
                    bit_cnt <= bit_cnt + 1'b1;
                end
            end
            else
            begin
```

```
                    sample_cnt <= sample_cnt + 1'b1;
                end
            end
            STOP:
            begin
                if(sample_cnt == 4'd8)
                begin
                    if(rx_in == 1'b1)
                    begin
                        data_out <= shift_reg;
                        ready    <= 1'b1;
                    end
                    sample_cnt <= 0;
                    state <= IDLE;
                end
                else
                begin
                    sample_cnt <= sample_cnt + 1'b1;
                end
            end
            default:
            begin
                state <= IDLE;
            end
            endcase
        end
    end
end
endmodule
```

## 13.4 uart_top.v

```
`timescale 1ns / 1ps
module uart_top(
    input clk,
    input reset,
    input wr_en,
    input [7:0] d_in,
    output tx_serial_out,
    output busy,
    output [7:0] data_out,
    output ready
);
wire tx_enb;
wire rx_enb;
baud_rate_generator baud_gen(
    .clk(clk), .reset(reset), .tx_enb(tx_enb), .rx_enb(rx_enb)
);
transmitter uart_tx(
    .clk(clk), .reset(reset), .wr_en(wr_en), .tx_enb(tx_enb),
    .d_in(d_in), .tx_serial_out(tx_serial_out), .busy(busy)
);
```

```
receiver uart_rx(
    .clk(clk), .reset(reset), .rx_enb(rx_enb), .rx_in(tx_serial_out),
    .data_out(data_out), .ready(ready)
);
endmodule
```

### 13.5 uart_tb.v

```
`timescale 1ns / 1ps
module uart_tb;
reg clk;
reg reset;
reg wr_en;
reg [7:0] d_in;
wire tx_serial_out;
wire busy;
wire [7:0] data_out;
wire ready;
uart_top DUT (
    .clk(clk), .reset(reset), .wr_en(wr_en), .d_in(d_in),
    .tx_serial_out(tx_serial_out), .busy(busy), .data_out(data_out), .ready(ready)
);
initial begin
    clk = 1'b0;
    forever #5 clk = ~clk;
end
initial begin
    reset = 1'b1; wr_en = 1'b0; d_in = 8'h00;
    #100; reset = 1'b0;
    #100; d_in = 8'hA5; wr_en = 1'b1; #10; wr_en = 1'b0;
    wait(ready);
    if(data_out == 8'hA5) $display("PASS : A5 Received");
    else $display("FAIL : Expected A5 Received %h",data_out);
    #1000; d_in = 8'h3C; wr_en = 1'b1; #10; wr_en = 1'b0;
    wait(ready);
    if(data_out == 8'h3C) $display("PASS : 3C Received");
    else $display("FAIL : Expected 3C Received %h",data_out);
    #1000; d_in = 8'h81; wr_en = 1'b1; #10; wr_en = 1'b0;
    wait(ready);
    if(data_out == 8'h81) $display("PASS : 81 Received");
    else $display("FAIL : Expected 81 Received %h",data_out);
    #1000; d_in = 8'h55; wr_en = 1'b1; #10; wr_en = 1'b0;
    wait(ready);
    if(data_out == 8'h55) $display("PASS : 55 Received");
    else $display("FAIL : Expected 55 Received %h",data_out);
    #5000;
    $display("UART LOOPBACK TEST COMPLETED");
    $finish;
end
endmodule
```